

โครงงาน

Project Game Object-oriented

PhuYaiBan

จัดทำโดย

6704062611280 นายภูมิพัฒน์ จันทร์ทิวากุล

เสนอ

ผู้ช่วยศาสตราจารณ์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาลัยเทคโนโลยีและสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

บทที่ 1 บทนำ

ที่มาและความสำคัญของโปรเจ็ค

เกมแนวเลื่อนฉากข้างเป็นรูปแบบที่ผู้เผู้เล่นคุ้นเคย และเหมาะกับการเรียรีนรู้การเขียนโปรแกรมเชิงวัตวัถุ (OOP) เพราะมีตัวละครหลายชนิดที่ต้องจัดการพร้อมกัน ได้แก่ ผู้ล่น ศัตรู กระสุน และไอเท็ม ซึ่งแต่ละอย่างมี พฤติกรรมของตัวเอง แต่ใช้โครงสร้างร่วมกันได้ โครงการนี้จึงสร้างเกมจริงที่เล่นได้ เพื่อนำ หลักการ OOP ที่ เรียนมาไปใช้ตั้งแต่การออกแบบคลาส การสืบทอด การวาดหน้าจอด้วย GUI ไปจนถึงการจัดการเหตุการณ์จาก แป้นพิมพิมพ์ และเมาส์ เนื้อเรื่อใช้ธีม ไทย ให้ผู้ใหญ่บ้านต่อสู้กับผีที่ เป็นภาพจำ ของวัฒณธรรมไทย เช่นช่ ผีผ้าขาว ผีโผีขมด (ผีไฟลอย) และผีตาโขน โดยออกแบบให้น่ารักไม่รุนแรง เพื่อให้เล่นได้ทุกทุกวัย

ประเภทของโครงการ

เกมคอมพิวพิเตอร์ 2 มิติแนว Side-Scrolling Shooter / Platformer
Desktop Application (ไม่ใม่ Applet) เปิดเป็นหน้าต่างขนาด 960×540 พิกเซล
เล่นคนเดียว มี 5 ด่าน ความยากเพิ่มขึ้นตามลำดับ

ประโยชน์

1. ได้ฝึกใช้หลัก OOP ในงานจริงริ ได้แก่ Inheritance, Abstract, Polymorphism, Encapsulation และ Composition
2. ได้ฝึกเขียน GUI ด้วย Swing และจัดการ Event จากคีย์บอดร์ และเมาส์
3. ได้เรียนรู้เทคนิคพื้นฐานของเกม เช่นช่ game loop, การชนกัน (collision), กล้องที่เลื่อนตามผู้เผู้ล่น และปัญญาประดิษฐ์แบบง่ายของศัตรู
4. ได้ฝึกแยกข้อมูลออกจากโค้ด โดยออกแบบด่านเป็นไฟล์ข้อความที่แก้ได้โดยไม่ต้องคอมไพล์ใหม่
5. ได้ฝึกใช้ Git/GitHub ควบคุมเวอร์ชันและแพ็กงานเป็นไฟล์ .jar แจกจ่ายได้
6. ผู้เผู้ล่นได้ความบันบันเทิงและรู้จักภาพผีไทยในรูปแบบที่ไม่น่ากลัว

ขอบเขตของโครงการ(แนบ Proposal)

1.4 ขอบเขตของโครงการ

สิ่งที่โปรแกรมทำได้

- เมนูหลักมีปุ่ม Play และ Quit และมีหน้าจอ Game Over / Win พร้อมปุ่มเล่นใหม่และกลับเมนู
- ผู้เล่นเดินซ้าย-ขวา กระโดด พุ่ง (dash) และยิงกระสุนไปตามทิศทางเมาส์ มีหลอดเลือดและหลอดแสดงการพุ่ง
- ศัตรู 3 แบบ: ศัตรูเดิน (Walker) ศัตรูบิน (Flyer) ที่ยิงกระสุนได้ และแท็งก์ (Tank) ที่อึดและพุ่งชน นอกจากนี้ Walker สามารถยกโล่สะท้อนกระสุนของผู้เล่นได้
- ไอเท็มฮีล (ดอกบัว) เก็บแล้วฟื้นเลือด
- 5 ด่าน ออกแบบแผนที่และตำแหน่งศัตรูเป็นไฟล์ข้อความ และปรับความยากด้วยค่า difficulty ของแต่ละด่าน
- เสียงประกอบสังเคราะห์ในตัวโปรแกรม ปิดเสียงได้และปรับความดังได้

สิ่งที่อยู่นอกขอบเขต

- ไม่มีโหมดผู้เล่นหลายคน ไม่มีระบบบันทึกเกม และไม่มี Applet
- ศัตรูทำงานเฉพาะตอนอยู่ในหน้าจอ

Proposal

Phu Yai Ban

Story - จู่ๆกลางคืนวันหนึ่งมีผีออกมาถล่มคนในหมู่บ้าน แต่เราออกจากหมู่บ้านพอดีเราจึงฝ่าดงผีเพื่อกลับไปช่วยครอบครัว



Gameplay - เกมยิงโดยใช้ A D เดินซ้ายขวา และ spacebar กระโดด คลิกซ้ายยิง
เราต้องวิ่งไปถึงจุดหมายให้ได้โดยกำจัดอุปสรรคที่เข้ามาด่านมีทั้งหมด 2 ด่าน





ประโยชน์ - ได้ใช้ไหวพริบชนะอุปสรรค
ตารางแผนงาน - เดือนกันยายน

ลำดับ	รายการ	1-15	16-25	26-30
1	หารูปตัวละครและทำกราฟิกต่างๆ			
2	ศึกษาเอกสารและข้อมูลที่เกี่ยวข้อง			
3	ลงมือเขียนโปรแกรม			
4	จัดทำเอกสาร			
5	ตรวจสอบและแก้ไขข้อผิดพลาด			

บทที่ 2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่องย่อ: ผู้ใหญ่บ้านออกเดินทางผ่าน 5 ด่าน เพื่อปราบผีที่มารบกวนหมู่บ้าน ตั้งแต่ทางเข้าหมู่บ้าน สะพานที่พัง ถ้ำ ไปจนถึงป้อมปลายทาง แต่ละด่านมีธงปลายทาง แตะธงเมื่อไรก็ผ่านด่าน ผีไม่ได้ทำร้ายจริงจัง แค่แกล้งให้ตกใจ ผู้ใหญ่บ้านจึงใช้ปืนน้ำมนต์ไล่พวกมันไปทีละตัว

เป้าหมาย: เดินไปถึงธงปลายทางในแต่ละด่านโดยไม่ตาย ผ่านครบ 5 ด่านจะชนะ เลือดหมดหรือตกหลุมจะแพ้

ปุ่มควบคุม

ปุ่ม	หน้าที่
A / D หรือ ลูกศรซ้าย-ขวา	เดินซ้าย / ขวา
Space, W หรือ ลูกศรขึ้น	กระโดด (กดค้าง = สูงขึ้น, ปล่อยเร็ว = เตี้ย)
Shift	พุ่ง (dash) ระยะประมาณ 140 พิกเซล หลบการโจมตีได้ระหว่างพุ่ง รอ 0.75 วินาทีก่อนพุ่งอีกครั้ง
เมาส์ซ้าย	ยิงกระสุนไปทางเมาส์ กดค้างเพื่อยิงรัว (ประมาณ 5 นัดต่อวินาที)
M	ปิด/เปิดเสียง
- / =	ลด / เพิ่มความดังเสียงทีละ 10%
ESC	กลับไปเมนูหลัก
F3 / F5	แสดง FPS / โหลดไฟล์ด่านใหม่ทันที (สำหรับผู้พัฒนา)

กติกาของเกม

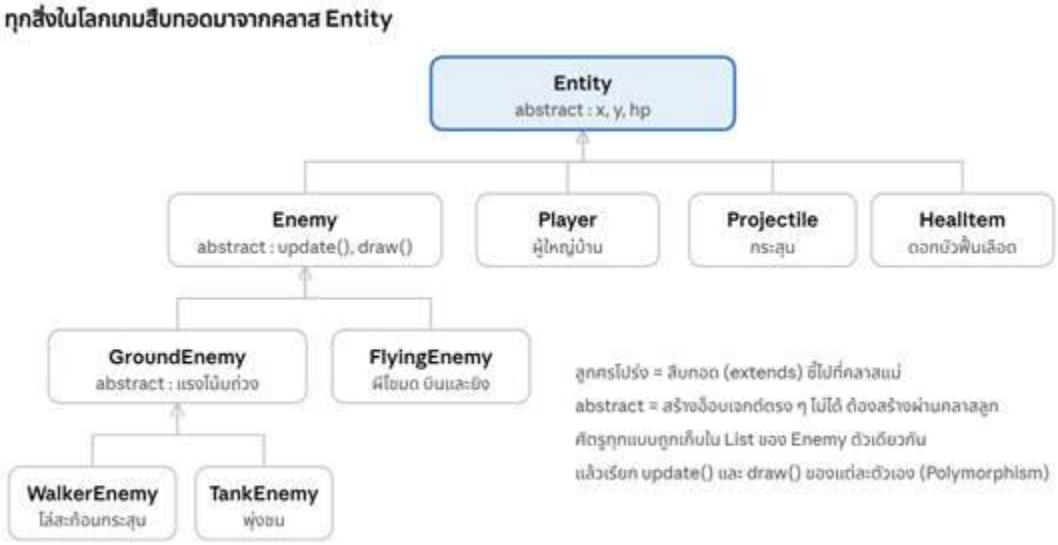
- ผู้เล่นเริ่มที่เลือด 100 หลังโดนตีจะอมตะ 1 วินาที ผ่านด่านได้โบนัส 500 คะแนนและฟื้นเลือด 50 ก่อนเข้าด่านถัดไป
- ไอเท็มดอกบัวฟื้นเลือด 30 เก็บได้เมื่อเลือดยังไม่เต็ม
- ตกหลุมในฉากจะตายทันที
- ความยากของศัตรูเพิ่มตามด่าน (ด่าน 1 คูณ 1.2 ไปจนถึงด่าน 5 คูณ 2.5) ส่งผลต่อเลือด ความเร็ว และดาเมจของศัตรู

ศัตรูแต่ละชนิด (ค่าเลือดและดาเมจคูณด้วย difficulty ของด่าน)

ศัตรู	จริงคือ	เลือดพื้นฐาน	ดาเมจชน	คะแนน	พฤติกรรม
Walker	ผีห่อผ้าขาว	30	15	100	เดินเข้าหากระโดดตามเมื่อผู้เล่นอยู่สูง ไม่เดินตกหลุม ยกโล่สะท้อนกระสุน
Flyer	ผีโขมด	20	10	150	ลอยยิงกระสุน คงระยะห่าง ส่ายไหวหลบกระสุนของผู้เล่น
Tank	ผีตาโขน	120	15 (ตอนพุ่ง 30)	300	เดินช้า เตรียมตัวแล้วพุ่งชน พุ่งชนกำแพงจะมึนและเปิดช่องให้โจมตี

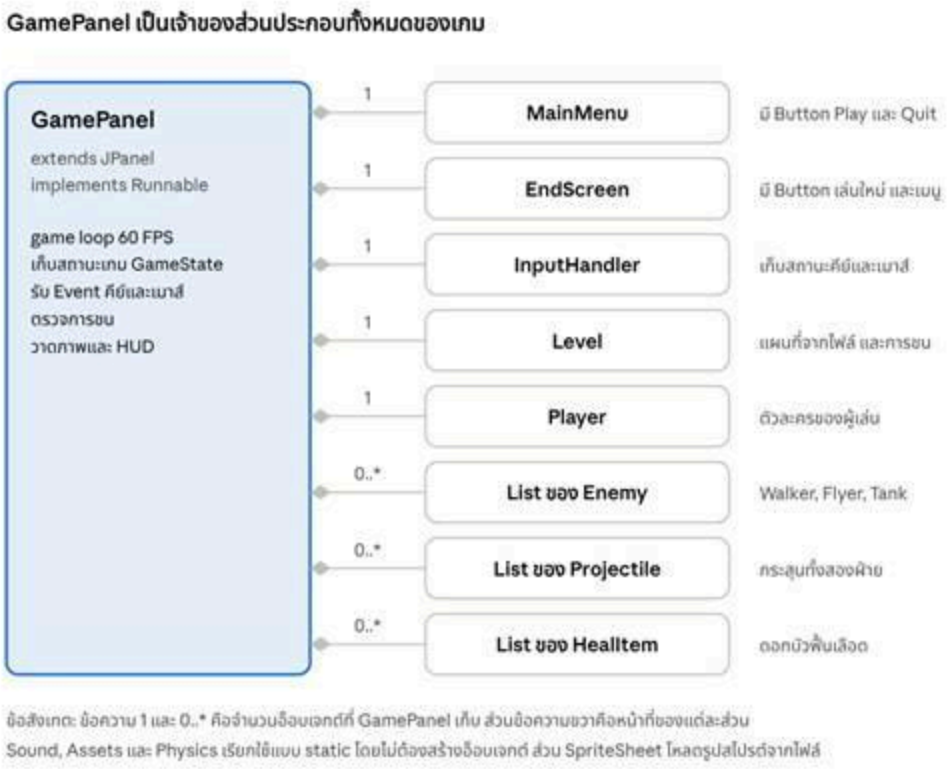
2.2 คลาสไดอะแกรมและคำอธิบาย

โปรแกรมแบ่งเป็น 8 แพ็กเกจ 22 ไฟล์ รวมประมาณ 2,900 บรรทัด แยกงานตามหน้าที่ แผนภาพด้านล่างแสดงความสัมพันธ์สองแบบ คือ การสืบทอด (extends) ในกลุ่ม entity และการประกอบ (Composition) ของ GamePanel



แผนผังการสืบทอดของคลาสในแพ็กเกจ entity

คลาส Entity เป็นแม่ของผู้เล่น ศัตรู กระสุน และไอเท็ม ศัตรูสืบทอดต่อผ่าน Enemy และ GroundEnemy ตามลำดับ



การประกอบ (Composition) ของ GamePanel

GamePanel ไม่ได้สืบทอดจากคลาสเหล่านี้ แต่สร้างและถือเป็นฟิลด์ของตัวเอง จึงเป็นความสัมพันธ์แบบ Composition

แพ็กเกจ	คลาส	หน้าที่
main	Main	สร้างหน้าต่าง JFrame ขนาด 960×540 และเริ่มเกม
main	GamePanel	หัวใจของโปรแกรม: game loop, สถานะเกม, รับ Event, ตรวจการชน, วาดภาพและ HUD
state	GameState	enum สถานะเกม: MENU, PLAYING, GAME_OVER, WIN
entity	Entity (abstract)	คลาสแม่ของทุกสิ่งในโลกเกม เก็บตำแหน่ง ขนาด เลือด และกรอบชน
entity	Player	ผู้ใหญ่บ้าน: เดิน กระโดด พุ่ง ยิง และวาดตัวเอง

entity	Enemy (abstract)	แม่แบบของศัตรูทุกแบบ กำหนด update() และ draw() ให้ลูกทำเอง
entity	GroundEnemy (abstract)	ศัตรูบนพื้น: แรงโน้มถ่วง กระโดด ตรวจไม่ให้เดินตกหลุม
entity	WalkerEnemy	ผีห่อผ้าขาว: เดินเข้าหา กระโดดตาม ยกโล่สะท้อนกระสุน
entity	TankEnemy	ผีตาโขน: อึด เตรียมตัวแล้วพุ่งชน
entity	FlyingEnemy	ผีโขมด: บิน ส่ายหลบ ยิงกระสุนเล็งผู้เล่น
entity	Projectile	กระสุนของทั้งผู้เล่นและศัตรู สะท้อนกลับได้
entity	HealItem	ดอกบัวฟื้นเลือด
level	Level	โหลดไฟล์ด่าน สร้างศัตรูและไอเท็ม ตอบคำถามว่าช่องไหนตัน และวาดแผนที่
level	Physics	ชนกับบล็อกแบบแยกแกน X และ Y
input	InputHandler	เก็บสถานะคีย์บอร์ดและเมาส์ (extends KeyAdapter)
ui	MainMenu, EndScreen	หน้าเมนูหลัก และหน้า Game Over / Win
ui	Button, HealthBar	ปุ่มที่คลิกได้ และหลอดเลือด
audio	Sound	เสียงสั้งเคราะห์ในโค้ด ปิด/เปิดและปรับความดัง
util	Assets, SpriteSheet	หาไฟล์ใน res และตัดเฟรมจากแผ่นสไปรต์

2.3 รูปแบบโปรแกรมและหลักการ OOP

รูปแบบ: เป็น **Application** (ไม่ใช่ Applet) เริ่มทำงานที่ main() ในคลาส Main ซึ่งสร้าง JFrame ขนาดคงที่ 960×540 แล้วใส่ GamePanel เข้าไป เกมทำงานใน thread แยก (game loop) และวาดภาพผ่าน paintComponent() ผลงานสุดท้ายแพ็กเป็นไฟล์ .jar ที่รวมฟส์ด่าน รูป ไว้ในตัว

Constructor

ทุกคลาสสร้างอ็อบเจกต์ผ่าน constructor และส่งค่าเริ่มต้นไปให้คลาสแม่ด้วย super(...) เช่น WalkerEnemy ส่งตำแหน่ง ขนาด เลือด ดาเมจ และคะแนนไปให้ GroundEnemy และ Enemy ตามลำดับ และคูณเลือดด้วยค่า difficulty ของด่าน

นอกจากนี้ WalkerEnemy มี constructor 2 แบบ (Constructor Overloading) แบบแรกไม่มีสกิลสะท้อนกระสุน เรียกแบบที่สองต่อด้วย this(...)

```
public WalkerEnemy(double centerX, double feetY, double difficulty) {
        this(centerX, feetY, difficulty, false);
}

 public WalkerEnemy(double centerX, double feetY, double difficulty, boolean reflect) {
        super(centerX - 20, feetY - 50, 40, 50,
        (int) Math.round(30 * difficulty),   // เลือด
        (int) Math.round(15 * difficulty),   // ดาเมจตอนชน
        100);                                // คะแนน
        this.speed = 1.8 * difficulty;
        this.reflectEnabled = reflect;
}
```

Encapsulation

ซ่อนข้อมูลภายในด้วย private และให้คลาสอื่นใช้ผ่านเมธอดสาธารณะเท่านั้น ตัวอย่าง:

- **Button** เก็บ bounds, label, onClick เป็น private final คลาสอื่นทำได้แค่เรียก mouseMoved(), mousePressed() และ draw()
- **Player** ซ่อนความเร็วแนวดิ่ง ตัวนับแดช และคูลดาวน์การยิงไว้เป็น private ภายออกถามสถานะผ่าน isDashing(), dashReadyRatio() และสั่งเปลี่ยนเลือดผ่าน takeDamage(), heal()
- **Sound** ซ่อนค่าความดัง volume เป็น private แก้ผ่าน changeVolume() ซึ่งจำกัดค่าให้อยู่ระหว่าง 0.0 ถึง 1.0 เสมอ

```
public static void changeVolume(double delta) {
        volume = Math.max(0.0, Math.min(1.0, Math.round((volume + delta) * 10) / 10.0));
}
```

สิ่งที่ควรปรับปรุง: คลาส Entity เปิดฟิลด์ x, y, hp เป็น public เพื่อให้เขียนโค้ดการชนและคำนวณตำแหน่งได้สะดวก แต่หากทำให้สมบูรณ์ตามหลัก Encapsulation ควรเปลี่ยนเป็น private พร้อม getter/setter และให้เลือดลดผ่าน takeDamage() เท่านั้น

Composition

คลาสใหญ่ประกอบขึ้นจากอ็อบเจกต์ของคลาสเล็ก (ความสัมพันธ์ has-a) อ็อบเจกต์เหล่านั้นถูกสร้างและใช้งานภายในเจ้าของ (ดูแผนภาพในหัวข้อ 2.2)

- **GamePanel** มี MainMenu, EndScreen, InputHandler, Level, Player และลิสต์ของ Enemy, Projectile, HealItem
- **Level** มีแผนที่ char[][] tiles และรายการจุดเกิดศัตรู List<Spawn>

```
private final InputHandler input = new InputHandler();
 private volatile Level level;
 private volatile Player player;
 private final List<Enemy> enemies = new CopyOnWriteArrayList<>();
```

Polymorphism

ศัตรูทุกแบบถูกเก็บในลิสต์ชนิด Enemy เดียวกัน GamePanel เรียก update() และ draw() โดยไม่ต้องรู้ว่าเป็นศัตรูชนิดไหน Java เลือกเมธอดของคลาสจริงตอนรัน (Method Overriding) ทำให้ Walker เดิน Flyer บิน และ Tank พุ่ง จากโค้ดชุดเดียวกัน และเพิ่มศัตรูชนิดใหม่ได้โดยไม่ต้องแก้ GamePanel

```
// จาก GamePanel (ตัดย่อ)
 for (Enemy en : enemies) {
        if (en.isOnScreen(cameraX)) {
        en.update(player, level, projectiles);
        } else {
        en.updateOffScreen(level);
        }
 }
```

ตัวอย่าง override อื่น: TankEnemy.getContactDamage() คืนดาเมจแรงขึ้นตอนพุ่ง และ WalkerEnemy.tryReflect() ที่คลาสแม่ Enemy คืน false เป็นค่าเริ่มต้น (ศัตรูอื่นไม่สะท้อน) แต่ Walker แก้ให้คืน true เมื่อโล่ยกอยู่และกระสุนพุ่งเข้าหน้า

```
@Override
 public int getContactDamage() {
        return state == State.CHARGE ? chargeDamage : contactDamage;
 }
```

ใน GamePanel เรียก en.getContactDamage() และ en.tryReflect(p) ทุกครั้งโดยไม่เช็กชนิดของศัตรู ส่วน Constructor Overloading ที่กล่าวไว้ข้างต้นเป็น Polymorphism อีกแบบที่เกิดตอนคอมไพล์

Abstract

คลาสนามธรรมในโปรแกรมมี 3 คลาส:

1. **Entity** สร้างอ็อบเจกต์ตรง ๆ ไม่ได้ เก็บข้อมูลร่วม (ตำแหน่ง ขนาด เลือด) และเมธอด getBounds(), isAlive()
2. **Enemy** บังคับให้ลูกทุกคนเขียน update() และ draw() เอง
3. **GroundEnemy** รวบรวมสิ่งที่ศัตรูบนพื้นใช้ร่วมกัน เช่น applyGravity(), jump(), wouldWalkIntoVoid() ให้ Walker และ Tank ใช้ซ้ำ ไม่ต้องเขียนสองรอบ

```
public abstract class Enemy extends Entity {
        public abstract void update(Player player, Level level, List<Projectile> projectiles);
        public abstract void draw(Graphics2D g2, int cameraX);
 }
```

Inheritance

คลาสลูกรับฟิลด์และเมธอดจากคลาสแม่ได้เลย ไม่ต้องเขียนซ้ำ โครงสร้างการสืบทอดทั้งหมด:

คลาสลูก	**สืบทอดจาก**	**สิ่งที่ได้รับมา**
Player, Projectile, HealItem, Enemy	Entity	**ตำแหน่ง ขนาด เลือด กรอบชน**
GroundEnemy, FlyingEnemy	Enemy	**ดาเมจชน คะแนน การวาดหลอดเลือด และพฤติกรรม ประจำจอ**
WalkerEnemy, TankEnemy	GroundEnemy	**แรงโน้มถ่วง การกระโดด การตรวจหลุม**
InputHandler	KeyAdapter (Java)	**รับ Event คีย์บอร์ด**
GamePanel	JPanel (Java)	**พื้นที่วาดภาพ**

```
public class WalkerEnemy extends GroundEnemy {
        public WalkerEnemy(double centerX, double feetY, double difficulty, boolean reflect) {
        super(centerX - 20, feetY - 50, 40, 50, ...);   // ส่งต่อให้คลาสแม่
        }
}
```

2.4 หน้าจอ GUI และส่วนประกอบ

โครงสร้าง GUI เบามาก มี Swing Component เพียง 2 ตัว คือ JFrame (กรอบหน้าต่าง) และ GamePanel extends JPanel (พื้นที่เกมทั้งหมด) ส่วนปุ่ม หลอดเลือด และข้อความทั้งหมดวาดเองด้วย Graphics2D ใน paintComponent() เพราะเกมต้องควบคุมทุกพิกเซลและวาดซ้ำ 60 ครั้งต่อวินาที

ส่วนประกอบ	ชนิด	หน้าที่
JFrame “My Game”	Swing Component	หน้าต่างหลัก ขนาดคงที่ (setResizable false) ปิดแล้วจบโปรแกรม (EXIT_ON_CLOSE) อยู่กลางจอ
GamePanel	JPanel	พื้นที่วาด 960×540 พิกเซล เปิด double buffering รับคีย์และเมาส์
MainMenu	วาดเอง	เมนูหลัก ปุ่ม PLAY และ QUIT
EndScreen	วาดเอง	หน้า GAME OVER / YOU WIN แสดงคะแนน พร้อมปุ่มเล่นใหม่และกลับเมนู
Button	วาดเอง	ปุ่มที่เปลี่ยนสีเมื่อเมาส์ชี้ (hover) และสั่งงานเมื่อคลิก
HealthBar	วาดเอง	หลอดเลือดผู้เล่น เปลี่ยนสีเขียว เหลือง แดง ตามปริมาณเลือด พร้อมตัวเลข HP
HUD	วาดใน drawHud()	หลอดเลือด หลอดความพร้อมพุ่ง คะแนน ชื่อด่าน และคำแนะนำปุ่ม
จุดเล็ง (crosshair)	วาดเอง	แสดงตำแหน่งเมาส์บนจอเมื่อกำลังเล่น

การวาดภาพซ้อนชั้น (ไล่จากหลังไปหน้า): ฟ้าและภูเขาไกล ๆ (parallax) → แผนที่พื้นและแพลตฟอร์ม → ไอเท็ม → ศัตรู → กระสุน → ผู้เล่น → HUD และจุดเล็ง กล้องเลื่อนตามผู้เล่นโดยลบค่า cameraX ออกจากพิกัดโลกก่อนวาด

สถานะของหน้าจอ: ควบคุมด้วย enum GameState 4 ค่า คือ MENU (แสดง MainMenu), PLAYING (เล่นเกม), GAME_OVER และ WIN (แสดง EndScreen) GamePanel ใช้ switch (state) เลือกว่าจะ update และวาดหน้าไหน

2.5 Event handling

โปรแกรมใช้แนวคิด Event Listener ของ Java รับเหตุการณ์จากคีย์บอร์ดและเมาส์บน GamePanel แล้วเก็บผลไว้เป็นสถานะ (กดอยู่ไหม เมาส์อยู่ตำแหน่งไหน) ซึ่ง game loop อ่านไปใช้ทุกเฟรม ทำให้ตัวละครตอบสนองลื่นไหล แทนที่จะขยับตัวละครตรงใน listener

Event / Listener	ที่อยู่	สิ่งที่เกิดขึ้น
KeyListener (InputHandler extends KeyAdapter)	InputHandler	keyPressed / keyReleased เก็บสถานะ A/D/ลูกศร (เดิน) Space/W/ลูกศรขึ้น (กระโดด) และ Shift (พุ่ง)
KeyListener (KeyAdapter ใน GamePanel)	GamePanel	ESC กลับเมนู, M ปิดเสียง, - / = ปรับความดัง, F3 แสดง FPS, F5 โหลดด่านใหม่
MouseMotionListener: mouseMoved, mouseDragged	GamePanel	อัปเดตตำแหน่งเมาส์ ใช้เล็งปืนและให้ปุ่มเปลี่ยนสีตอนชี้
MouseListener: mousePressed, mouseReleased	GamePanel	ในเมนูและหน้าจบเกม = คลิกปุ่ม ในเกม = กดค้างเพื่อยิง ปล่อย = หยุดยิง
ปุ่ม (callback)	Button	เก็บ Runnable onClick เมื่อคลิกโดนปุ่มจะเรียก onClick.run()
ปิดหน้าต่าง	Main	EXIT_ON_CLOSE จบโปรแกรม

InputHandler (เก็บสถานะคีย์)

```
if (key == KeyEvent.VK_A || key == KeyEvent.VK_LEFT)  left = down;
 if (key == KeyEvent.VK_D || key == KeyEvent.VK_RIGHT) right = down;
 if (key == KeyEvent.VK_SPACE || key == KeyEvent.VK_W || key == KeyEvent.VK_UP) jump = down;
 if (key == KeyEvent.VK_SHIFT && down) dashPressed = true;
```

ปุ่มในเมนูส่งงานผ่าน callback เมนูไม่รู้ว่า Play หรือ Quit ต้องทำอะไร GamePanel เป็นผู้ส่งงานให้ตอนสร้าง ทำให้ Button นำกลับไปใช้ได้ทุกที่

```
menu = new MainMenu(
        this::startGame,                // Play
        () -> System.exit(0)            // Quit
 );
```

หมายเหตุเรื่อง thread: Event ทำงานบน Event Dispatch Thread ส่วน game loop ทำงานบน thread แยก จึงใช้ volatile กับฟิลด์ที่ทั้งสองฝ่ายแก้ (เช่น state, level, player) และใช้ CopyOnWriteArrayList กับลิสต์ศัตรูและกระสุน เพื่อไม่ให้เกิด ConcurrentModificationException เวลา thread วาดภาพอ่านลิสต์ในขณะที่ game loop กำลังแก้

2.6 อัลกอริทึมสำคัญในโปรแกรม

1) Game loop แบบเฟรมคงที่ 60 FPS

ใน thread แยกจะสะสมเวลาที่ผ่านไปเป็นหน่วยเฟรม (delta) เมื่อครบ 1 เฟรมจะเรียก update() และ repaint() แล้วลบ 1 ออก ทำให้เกมวิ่งความเร็วเท่ากันทุกเครื่อง ส่วน Thread.sleep(1) ช่วยไม่ให้ CPU เต็ม 100%

```
delta += (now - lastTime) / drawInterval;   // drawInterval = 1,000,000,000 / 60 นาโนวินาที
 if (delta >= 1) {
        update();
        repaint();
        delta--;
 }
```

2) กล้องเลื่อนตามผู้เล่น (Camera scrolling)

โลกกว้างกว่าจอมาก (ด่านยาวถึง 6,400 พิกเซล) จึงเก็บพิกัดทุกอย่างเป็นพิกัดโลก แล้วเลื่อนกล้อง cameraX ให้ผู้เล่นอยู่กลางจอ และจำกัดไม่ให้เกินขอบด่าน ตอนวาดจะลบ cameraX ออกจากพิกัดโลก และเมื่อแปลงตำแหน่งเมาส์กลับเป็นพิกัดโลกจะบวก cameraX

```
cameraX = (int) player.centerX() - WIDTH / 2;
 cameraX = Math.max(0, Math.min(cameraX, level.worldWidth - WIDTH));
```

3) เล็งและยิงกระสุนไปตามเมาส์

ใช้ Math.atan2 หามุมจากตัวผู้เล่นไปยังเมาส์ (ในพิกัดโลก) แล้วสร้าง Projectile ที่บินไปทางนั้น พร้อมคูลดาวน์ 12 เฟรมระหว่างนัด (ประมาณ 5 นัดต่อวินาที) ตัวละครหันไปทางเมาส์ด้วย

```
double mouseWorldX = input.mouseX + cameraX;
 aimAngle = Math.atan2(input.mouseY - centerY(), mouseWorldX - centerX());
 facingRight = mouseWorldX >= centerX();
```

4) ชนกับแผนที่แบบแยกแกน (Tile collision)

แผนที่เป็นตารางช่องขนาด 40×40 พิกเซล การขยับทำทีละแกน: ขยับแนวนอน (Physics.moveX) แล้วฝังชนบล็อกที่ทับอยู่ได้แก่หยุดชิดขอบบล็อก จากนั้นขยับแนวตั้ง (Physics.moveY) แล้วส่งผลกลับเป็น LANDED (ยืนพื้น) หรือ HIT_CEILING (หัวชนเพดาน) การแยกแกนทำให้ไถลกำแพงได้ไม่ติดมุม แพลตฟอร์มบาง (=) ผ่านจากด้านล่างและด้านข้างได้ แต่ยืนได้เมื่อตกลงมาจากด้านบน

```
Physics.moveX(this, vx, level);          // 1) แนวนอน
 int hit = Physics.moveY(this, velocityY, level);   // 2) แนวตั้ง
 if (hit == Physics.LANDED) { onGround = true; velocityY = 0; }
```

5) การกระโดด แรงโน้มถ่วง และพุ่ง

ความเร็วแนวตั้งเพิ่มขึ้นทุกเฟรมด้วยแรงโน้มถ่วง 0.6 (จำกัดความเร็วตกสูงสุด 15) กระโดดให้ความเร็วขึ้น 12.5 จึงขึ้นได้สูงสุดประมาณ $12.5^2 \div (2 \times 0.6) \approx 130$ พิกเซล (ประมาณ 3 ช่อง) ถ้าปล่อยปุ่มกลางอากาศจะตัดความเร็วขึ้นเหลือ 4 ทำให้กระโดดเตี้ยได้ (variable jump) การพุ่ง (dash) เคลื่อน 14 พิกเซลต่อเฟรมนาน 10 เฟรม ระหว่างนั้นอมตะ และต้องรอ 45 เฟรมก่อนพุ่งอีกครั้ง

6) อ่านไฟล์ด่านเป็นตาราง (Data-driven level)

ด่านเป็นไฟล์ข้อความ ส่วนหัวไฟล์กำหนด name, theme, difficulty, walker_reflect ใต้เส้น --- คือแผนที่ 1 ตัวอักษรเท่า 1 ช่อง (# บล็อก, = แพลตฟอร์มบาง, P จุดเกิด, G ธง, W F T ศัตรู, H ไอเท็มฟื้นเลือด) โปรแกรมอ่านทีละบรรทัดเก็บเป็น char[][] tiles และจดจุดเกิดศัตรูไว้ในลิสต์ เพิ่มด่านใหม่ได้โดยสร้างไฟล์และเติมชื่อใน LEVEL_FILES ไม่ต้องแก้อัลกอริทึม

7) ความยากตามด่าน

ค่า difficulty ของด่านถูกส่งเข้า constructor ของศัตรูและคูณกับเลือด ความเร็ว และดาเมจ เช่น Walker เลือด = 30 × difficulty ความเร็ว = 1.8 × difficulty ด่าน 1 (1.2) Walker เลือด 36 ด่าน 5 (2.5) เลือด 75 และยกโล่ถี่ขึ้น

8) AI ของศัตรู

ศัตรูทุกตัวทำงาน (เดิน บิน ยิง) เฉพาะตอนอยู่ในจอ isOnScreen(cameraX) นอกจอจะยืนนิ่งและตกตามแรงโน้มถ่วงเท่านั้น ช่วยหลีกเลี่ยงการคำนวณที่ไม่จำเป็นและทำให้มอนสเตอร์ไม่ไล่ก่อนเข้าจอ

- **Walker (ผีห่อผ้าขาว):** เดินเข้าหาผู้เล่น กระโดดขึ้นเมื่อผู้เล่นอยู่สูงกว่าหรือมีกำแพงเตี้ย 1 ช่องขวาง และยกโล่สะท้อนกระสุนเมื่อเห็นกระสุนพุ่งเข้าหน้า
- **Flyer (ผีโขมด):** ลอยอยู่ระยะห่างจากผู้เล่น ส่ายไหวด้วยคลื่น sine หลบกระสุนของผู้เล่นโดยเคลื่อนในแนวตั้งฉากกับทิศกระสุน และยิงกระสุนเล็งไปที่ผู้เล่น
- **Tank (ผีตาโขน):** ทำงานเป็น state machine ดังภาพด้านล่าง ทิศพุ่งถูกล็อกตอนเริ่มเตรียมตัว ผู้เล่นจึงหลบข้างหรือแดชพ้นได้



สถานะของ TankEnemy · 4 สถานะวนซ้ำ

การไม่เดินตกหลุมของ GroundEnemy: ก่อนก้าวไปข้างหน้า จะสแกนคอลัมน์ข้างหน้าลงไปจนสุดแผนที่ ถ้าไม่เจอพื้นหรือแพลตฟอร์มเลยแปลว่าเป็นหลุม จะหยุดเดิน (แต่ถ้ามีพื้นรองรับในระดับที่ต่ำกว่า ก็เดินลงได้)

```
for (int r = startRow; r < rows; r++) {
        if (level.isSolid(col, r) || level.isOneWay(col, r)) {
        return false;   // มีพื้นรองรับ เดินได้
        }
}
 return true;                // ไม่มีพื้นเลย = หลุม
```

9) ตรวจการชนระหว่างวัตถุ

ทุกอย่างมีกรอบสี่เหลี่ยม (getBounds()) ใช้ Rectangle.intersects() ตรวจเฟรมละครั้ง กระสุนผู้เล่นชนศัตรู: ถ้าศัตรูสะท้อนได้ (tryReflect) กระสุนกลายเป็นของศัตรูและพุ่งกลับ มิฉะนั้นศัตรูเสียเลือดและผู้เล่นได้คะแนน กระสุนศัตรูหรือการชนตัวทำให้ผู้เล่นเสียเลือด แต่ถ้ากำลังอมตะ (หลังโดนตี 60 เฟรม หรือระหว่างพุ่ง) จะไม่เสียเลือด

บทที่ 3 สรุป

3.1 ปัญหาที่พบระหว่างการพัฒนา

ปัญหา	สาเหตุ	วิธีแก้
โหลดไฟล์ด่านไม่เจอ (ขึ้น level missing file)	โปรแกรมหาไฟล์จากโฟลเดอร์ที่สั่งรันเท่านั้น เมื่อเปิดจากที่อื่นจึงหาไม่เจอ	เขียน Assets ให้ค้นหลายที่ ได้แก่ classpath, โฟลเดอร์ปัจจุบัน, โฟลเดอร์ย่อย และโฟลเดอร์แม่จากที่ของโค้ด และแสดงรายชื่อที่ค้นแล้วไม่เจอให้แก้ง่าย
ศัตรูเดินตกหลุมเอง และตามผู้เล่นไม่ทันเมื่ออยู่สูงกว่า	ศัตรูเดินแค่ทิศเข้าหาผู้เล่น ไม่มองพื้นข้างหน้า	เพิ่ม wouldWalkIntoVoid() สแกนพื้นข้างหน้า และให้ Walker กระโดดได้
มอนสเตอร์ไล่ตามทั้งที่ยังอยู่นอกจอ	update ศัตรูทุกตัวทุกเฟรม	แยก update() กับ updateOffScreen() ให้ทำงานเฉพาะตัวที่อยู่ในจอ
หน้าจอกระตุกหรือเกิด error เรื่องลิสต์บางครั้ง	thread วาดภาพกับ game loop แก้ข้อมูลพร้อมกัน	ใช้ volatile และ CopyOnWriteArrayList
ด่านยากจนผ่านไม่ได้ หรือง่ายเกินไป	กระโดดได้สูงสุดประมาณ 3 ช่อง ถ้าวางหลุมกว้างเกินไปจะข้ามไม่ได้	คำนวณระยะกระโดดจากสูตรฟิสิกส์ก่อนออกแบบ: ช่องว่างไม่เกิน 3–4 ช่อง และจุดที่ต้องพุ่งให้เป็นทางเลือกที่ทำได้ และเพิ่มไอเท็มฟื้นเลือดในจุดสำคัญ
ย้ายโปรเจกต์เข้าโฟลเดอร์ที่ผูกกับ Git แล้ว error เต็มไปหมด	ส่งทั้งโฟลเดอร์ไปวาง ทำให้โฟลเดอร์ซ้อนกัน โค้ดที่ประกาศ package main; จึงหาไม่เจอ	ย้ายเฉพาะของข้างใน (src, res, MANIFEST.MF) ให้อยู่ชั้นนอกสุดของโปรเจกต์ แล้วล้างแคช Java ใน VS Code
ล้มเลิกการ push ขึ้น GitHub (Internal Server Error)	ปัญหาอยู่ที่เซิร์ฟเวอร์ฝั่ง GitHub ไม่ใช่โค้ด	รอสักครู่แล้ว push ใหม่ ไฟล์ยังปลอดภัยใน commit บนเครื่อง

ย้อนไฟล์กลับไป version เก่าใน Git ไม่สำเร็จ	พิมพ์คำสั่งใส่เครื่องหมาย < > ครอบรหัส commit ตามตัวอย่าง ทำให้ Git อ่านไม่ออก	ใส่รหัส commit เปล่า ๆ ไม่ใส่เครื่องหมาย หรือใช้ปุ่ม Revert ใน GitHub Desktop
ดับเบิลคลิก .jar แล้วขึ้น “A Java Exception has occurred” แต่รัน java -jar ผ่าน	Windows ใช้ Java คนละตัวกับที่ cmd ใช้ (เครื่องมีหลายเวอร์ชัน)	ทำไฟล์ Play.bat เรียก javaw -jar หรือเปลี่ยนโปรแกรมที่เปิด .jar ให้ชี้ไปที่ Java ตัวใหม่ และคอมไพล์ด้วย --release 11 เพื่อให้เครื่องอื่นเปิดได้

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. **ธีมไทยแบบไม่รุนแรง:** ผู้ใหญ่บ้านสู้กับผีห่อผ้าขาว ผีโขมด และผีตาโขน ทุกตัวหน้าตาน่ารัก ไม่มีภาพเลือด ไอเท็มฟื้นเลือดเป็นดอกบัว เล่นได้ทุกวัย
2. **ผีห่อผ้าขาวยกโล่สะท้อนกระสุนได้:** ผู้เล่นต้องคิด ยิงจากด้านหลัง ยิงตอนโล่ลง หรือยิงเฉียงสูง
3. **ผีตาโขนบอกท่าก่อนโจมตี:** มีช่วงเตรียมตัวและเครื่องหมายเตือน ผู้เล่นอ่านท่าและหลบได้ การพาคอบรอบทำให้เกมยากแต่ยุติธรรม
4. **ออกแบบด่านได้เองด้วยไฟล์ข้อความ:** พิมพ์แผนที่ด้วยตัวอักษร แก้แล้วกด F5 โหลดใหม่ทันทีโดยไม่ต้องคอมไพล์ โดยปรับความยากทั้งด่านได้ด้วยค่า difficulty
5. **ศัตรูทำงานเฉพาะตอนอยู่ในจอ และ AI หลายรูปแบบ:** เดินกระโดดไม่ตกหลุม บินส่ายหลบ และพุ่งชนแบบ state machine
6. **ไม่ต้องมีไฟล์เสียง:** เสียงทุกชนิดสังเคราะห์จากคลื่นในโค้ด (javax.sound.sampled) แต่ถ้าวางไฟล์ .wav ใน res/sounds ชื่อตรงกัน เกมจะใช้ไฟล์แทนได้ทันที สไปรต์ก็ทำแบบเดียวกัน ถ้าไม่มีไฟล์รูปก็ใช้สี่เหลี่ยมแทนได้ไม่ทำให้เกมค้าง
7. **ปรับความดังเสียงและปิดเสียงได้ในเกม:** ปุ่ม - = และ M

3.3 คำแนะนำสำหรับผู้สอนและน้อง ๆ รุ่นถัดไป

เนื้อหาที่อยากให้ผู้สอนอธิบายเพิ่มในคลาส (จุดที่ผู้จัดทำเห็นว่ายากที่สุดอยู่ในโปรเจกต์นี้)

- game loop และการทำงานหลาย thread (Event Dispatch Thread กับ game thread) รวมถึง volatile และ CopyOnWriteArrayList
- การวาดด้วย Graphics2D ใน paintComponent() และระบบพิกัดโลกกับพิกัดหน้าจอ
- ความแตกต่างระหว่าง Inheritance (is-a) กับ Composition (has-a) และความแตกต่างระหว่าง abstract class กับ interface

- การส่งงานผ่าน callback (Runnable, lambda, method reference) ใน Button
- การอ่านไฟล์จาก classpath เทียบกับจากโฟลเดอร์จริง และการแพ็ก .jar
- การชนแบบ AABB และ state machine

สิ่งที่ควรเพิ่มในหลักสูตร

- สอนการเขียน .gitignore และการใช้ Git ในงานจริง (commit, push, ย้อน version) เพราะเจอปัญหานี้บ่อยมาก
- ชี้แนะวิธีแจกจ่ายงานให้คนอื่นเล่นได้ เช่น เรื่องเวอร์ชัน Java และวิธีสร้าง .jar

คำแนะนำสำหรับน้อง ๆ รุ่นถัดไป

1. ทำทีละขั้น เริ่มจากหน้าต่างเปล่า ไปเมนู ตัวละครเดิน ยิง ศัตรู และด่าน และ commit ทุกครั้งที่ทำเสร็จหนึ่งขั้น เผื่อพังจะย้อนกลับได้
2. ออกแบบคลาสแม่ (เช่น Entity) ตั้งแต่ต้น จะเพิ่มศัตรูชนิดใหม่ได้ง่าย
3. แยกข้อมูล (ด่าน รูป เสียง) ออกจากโค้ด จะปรับแต่งได้ไวโดยไม่ต้องคอมไพล์ใหม่
4. พยายามทดสอบไฟล์ .jar บนเครื่องอื่นก่อนส่งงาน และคอมไพล์ด้วย --release ที่ต่ำพอให้เครื่องที่ใช้ Java เก่ากว่าเปิดได้

สิ่งที่ควรต่อยอดในอนาคต

- บอสปลายด่าน ระบบเลือกด่าน และบันทึกคะแนนสูงสุด
- โหมดเล่น 2 คน และการตั้งค่าปุ่มควบคุม
- ปรับ Entity ให้สมบูรณ์ตามหลัก Encapsulation (ฟิลด์ private + getter/setter)
- เขียน unit test ให้กับฟิสิกส์และการชน